

VirtuEx 성능 개선 감사 보고서 (8차)

VirtuEx Performance Improvement Audit Report (8th)

- 감사일: 2026-03-19
- 감사 범위: 프론트엔드 렌더링/Memoization, 서버 페이지네이션, 번들 최적화, 이미지 처리, WebSocket 최적화, DB 쿼리 패턴, 캐싱 전략, 네트워크 효율
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음

이전 감사 대비 수정 현황 (7차 → 8차)

이전 이슈	상태	비고
[API-M-01] 주문 내역 서버 사이드 심볼 필터 미사용	수정됨	백엔드 <code>getUserOrders()</code> 에 <code>symbol</code> 쿼리 파라미터 전달 확인, <code>order-proxy.controller.ts</code> 에 <code>@Query('symbol')</code> 추가
[API-M-02] 체결 내역 서버 사이드 <code>side/symbol</code> 필터 미사용	수정됨	<code>getUserTrades()</code> 에 <code>symbol</code> , <code>side</code> 쿼리 파라미터 전달 확인, <code>order-proxy.controller.ts</code> 에 <code>@Query('symbol')</code> , <code>@Query('side')</code> 추가
[IMG-M-01] TipTap Base64 → 서버 업로드 API	수정됨	<code>RichEditor.tsx</code> 에서 인증 사용자는 <code>POST /api/community/upload-image</code> 로 서버 업로드 후 URL 삽입, 실패 시 <code>data URL</code> 폴백
[PF-M-01] 커뮤니티 페이지 944줄 단일 컴포넌트	수정됨	<code>community/page.tsx</code> 가 235줄로 축소, <code>DiscussionsTab</code> , <code>StrategiesTab</code> , <code>TradersTab</code> 으로 <code>lazy-loaded</code> 코드 스플리팅 완료
[PF-H-01] 대시보드 AI 분석 뉴스 200건 과잉 조회	미수정	여전히 <code>limit: 200</code> 으로 전체 조회 후 24시간 내 50건 슬라이싱
[PF-M-04] <code>AssetList</code> <code>assetsVersion</code> <code>useMemo</code> 매 tick 실행	부분 수정	<code>assetsLenRef + assetsSymbolsRef</code> 도입으로 심볼 구성 변경 시에만 재계산. 단, <code>currentSymbols</code> <code>useMemo</code> 자체가 매번 <code>assets.map().join()</code> 실행
[BD-M-01] 대시보드 516줄 단일 컴포넌트	미수정	TODO 주석 유지, 516줄 단일 컴포넌트 그대로
[BD-L-01] <code>lucide</code> 아이콘 re-export	보류	트리셰이킹으로 실질 영향 최소

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 프론트엔드 렌더링/Memoization	0	1	2	1	4
B. API/네트워크 효율	0	1	2	0	3
C. 번들/코드 분할	0	0	1	1	2

D. 캐싱/메모리 관리	0	0	2	1	3
E. DB 쿼리/서버 성능	0	0	1	1	2
F. WebSocket/실시간 처리	0	0	1	1	2
합계	0	2	9	5	16

A. 프론트엔드 렌더링/Memoization — 4건

[PF-H-01] 대시보드/뉴스 AI 분석 — 뉴스 200건 전체 fetch 후 50건 슬라이싱 (High, 미수정)

현상: dashboard/page.tsx:178 에서 `api.get('/api/news', { params: { limit: 200 } })` 로 200건 전체를 가져온 후 24시간 내 50건만 사용. news/page.tsx:157 에서도 동일 패턴 반복. 200건의 JSON 응답 크기가 수 백 KB에 달하여 불필요한 네트워크 전송과 파싱 부하 발생 위치:

`frontend/src/app/(main)/dashboard/page.tsx:178` ,
`frontend/src/app/(main)/news/page.tsx:157` **영향:** 불필요한 네트워크 전송량(최대 4배 과잉), 서버 DB 쿼리 부하, 모바일 환경에서 파싱 지연 **권장:** 서버에서 `publishedAfter` 파라미터로 24시간 내 뉴스만 조회하거나, `limit`을 50으로 줄이고 서버 측 날짜 필터 적용 **심각도:** High

[PF-M-01] AssetList currentSymbols — assets.map().join() 매 렌더 실행 (Medium, 부분 수정)

현상: AssetList.tsx:98 에서 `currentSymbols = useMemo(() => assets.map(a => a.symbol).join(','), [assets])` . `assets` 참조는 WebSocket 틱마다 새로 생성되므로 매번 100개+ 심볼을 `map+join`. `assetsVersion` `useMemo` 내부에서 `currentSymbols` 와 이전 문자열을 비교하여 불필요한 재정렬을 방지하지만, `join` 자체의 연산은 매번 발생 위치: `frontend/src/components/market/AssetList.tsx:98-108` **영향:** 500ms마다 100+ 심볼의 `join` 및 문자열 비교 수행 **권장:** `assetsRef.current.length` 와 첫/마지막 심볼 비교 등 `O(1)` 비교로 대체하거나, 상위 컴포넌트에서 심볼 목록을 안정적 `ref`로 전달 **심각도:** Medium

[PF-M-02] 주문 페이지 — trades 전체 로드 후 분석 탭에서 반복 filter (Medium)

현상: orders/page.tsx 의 분석 탭에서 `trades` 배열에 대해 각 심볼별 `trades.filter()` 호출이 반복. `useTradeHistory()` 가 전체 체결 내역을 가져온 후 클라이언트에서 심볼별/side별 필터링과 집계를 수행하여 `O(N*M)` 복잡도 위치: `frontend/src/app/(main)/orders/page.tsx` (분석 탭 섹션) **영향:** 거래 내역 1000건+ 시 분석 탭 전환 지연 **권장:** 단일 순회에서 `symbolMap` 으로 `buyCount/sellCount`를 동시 집계하거나, 서버 측 집계 API 제공 **심각도:** Medium

[PF-L-01] PortfolioHistoryChart — totalValue prop 변경 시 전체 재계산 (Low)

현상: PortfolioHistoryChart.tsx:127-130 에서 `buildHistoryFromTransactions` 의 `useMemo` 의존성에 `totalValue` 가 포함. 포트폴리오 가격이 10초 폴링으로 변할 때마다 전체 거래 내역을 정렬하고 시계열 데이터를 재구성. 거래 내역은 변하지 않았는데 마지막 포인트만 업데이트하면 됨 위치:
`frontend/src/components/portfolio/PortfolioHistoryChart.tsx:127-130` **영향:** 거래 내역 200건+ 시 불필요한 배열 복사/정렬 반복 **권장:** 거래 내역 기반 시계열은 `transactions` 변경 시에만 계산하고, `totalValue` 변경 시에는 마지막 포인트만 교체 **심각도:** Low

B. API/네트워크 효율 — 3건

[API-H-01] 뉴스 필터 적용 시 200건 일괄 조회 후 클라이언트 페이지네이션 (High)

현상: news/page.tsx:80-81 에서 검색어나 날짜 필터가 적용되면 `FILTERED_FETCH_LIMIT = 200` 으로 한번에 200건을 조회한 후 클라이언트에서 필터링+페이지네이션 수행. `useNews` 훅에 `search` , `publishedAfter` 파라미터 미전달

위치: frontend/src/app/(main)/news/page.tsx:78-81 **영향:** 불필요한 대량 데이터 전송, 서버 쿼리 부하, 모바일 데이터 낭비 **권장:** 서버 API에 search, dateFilter 파라미터 추가하여 서버 측 필터링 적용, 클라이언트는 페이지 단위로만 요청 **심각도:** High

[API-M-01] 체결 내역 — 프론트엔드에서 서버 페이지네이션 미활용 (Medium)

현상: useTradeHistory() 혹은 hooks/useOrders.ts:194-219 이 /api/orders/trades/history 에 limit / offset 파라미터를 전달하지 않고 전체 체결 내역을 가져옴. 백엔드는 이미 limit / offset 지원을 확인했으나 프론트엔드에서 미활용 **위치:** frontend/src/hooks/useOrders.ts:197-198 **영향:** 체결 내역 누적 시 응답 크기 선형 증가 **권장:** useTradeHistory 에 page/limit 파라미터 추가, 커서 기반 무한 스크롤 적용 **심각도:** Medium

[API-M-02] 포트폴리오 valuation/summary — 동시 폴링 중복 (Medium)

현상: usePortfolio() 와 usePortfolioValuation() 이 각각 독립적으로 10초 폴링 설정. 포트폴리오 페이지에서 두 호출이 동시 활성화될 경우 거의 동일한 데이터를 2배로 요청. portfolio/page.tsx 에서 usePortfolioValuation 만 사용하지만, OrderForm.tsx 의 usePortfolio() 도 동시 활성화 가능 **위치:** frontend/src/hooks/usePortfolio.ts:102-117, 125-138 **영향:** 동일 사용자에게 대해 10초마다 서버 요청 2건 중복 **권장:** usePortfolioValuation 하나로 통합하거나, 쿼리 키를 공유하여 중복 방지 **심각도:** Medium

C. 번들/코드 분할 — 2건

[BD-M-01] 대시보드 페이지 — 516줄 단일 컴포넌트 (Medium, 미수정)

현상: dashboard/page.tsx 가 516줄로 실시간 가격, WebSocket 배치, AI 분석 모달, 스포트라이트 검색, 탭 상태, 관심종목 등 모든 로직을 포함. TODO 주석(RD-M-02)으로 분리 필요성이 명시되어 있으나 미수행. AI 분석 모달 코드(60줄+)가 사용되지 않아도 항상 로드됨 **위치:** frontend/src/app/(main)/dashboard/page.tsx:1-516 **영향:** 초기 번들 크기 증가, AI 분석 모달 미사용 시에도 관련 상태와 이벤트 핸들러가 초기화됨 **권장:** AI 분석 모달을 dynamic(() => import(...)) 으로 분리, WebSocket 배치 로직을 커스텀 훅으로 추출 **심각도:** Medium

[BD-L-01] 주문 페이지 — 인라인 분석 로직으로 번들 비대화 (Low)

현상: orders/page.tsx 가 거래 분석(심볼별 breakdown, 일별 통계, CSV 내보내기 등)과 주문 관리 로직을 모두 포함. 분석 탭 UI와 주문 목록 UI가 동일 번들에 포함되어 주문 목록만 조회할 때도 분석 코드가 로드됨 **위치:** frontend/src/app/(main)/orders/page.tsx **영향:** 초기 번들 크기 증가 (분석 탭 미사용 시 불필요한 코드) **권장:** 분석 탭을 dynamic(() => import(...)) 으로 분리하여 탭 전환 시에만 로드 **심각도:** Low

D. 캐싱/메모리 관리 — 3건

[CM-M-01] CandlestickChart — chartType 변경 시 8개 시리즈 전체 재생성 (Medium)

현상: CandlestickChart.tsx:319-404 에서 chartType 변경 시 기존 시리즈 전부 제거 후 재생성. 캔들 ↔ 라인 전환 시 메인 시리즈뿐 아니라 볼륨, SMA 3개,布林지 밴드 3개까지 총 8개 시리즈를 destroy/recreate. 이후 Effect 4에서 다시 데이터 세팅 **위치:** frontend/src/components/chart/CandlestickChart.tsx:319-404 **영향:** 차트 타입 전환 시 눈에 띄는 깜빡임과 지연 (시리즈 생성 → 데이터 세팅의 2단계 렌더) **권장:** 메인 시리즈만 교체하고 보조 지표 시리즈는 유지, 또는 두 시리즈를 모두 생성 후 visible 토글로 전환 **심각도:** Medium

[CM-M-02] 리더보드 sortedLeaderboard — investedOnly/copyTradeOnly 필터 변경 시 전체 재정렬 (Medium)

현상: leaderboard/page.tsx:212-230 의 sortedLeaderboard useMemo가 investedOnly, copyTradeOnly 등 6개 의존성을 가짐. 토크 변경마다 전체 배열을 filter → sort → map으로 재계산. 사용자 100명+ 시 불

필요한 정렬 반복 위치: `frontend/src/app/(main)/leaderboard/page.tsx:212-230` 영향: 필터 토글마다 전체 정렬 수행으로 UI 버벅임 권장: 정렬 결과를 캐싱하고 필터만 별도 useMemo로 분리하여 필터 변경 시 재정렬 없이 필터링만 수행
심각도: Medium

[CM-L-01] 리더보드 prevRankMap — 기간/정렬 변경 시 미초기화 (Low)

현상: `leaderboard/page.tsx:235` 의 `prevRankMap.current` 가 컴포넌트 생애 동안 이전 순위 데이터를 계속 유지. `period` 나 `sortMode` 변경 시에도 초기화되지 않아 잘못된 순위 변동 애니메이션 발생 가능 위치: `frontend/src/app/(main)/leaderboard/page.tsx:235` 영향: 필터 변경 후 순위 변동 인디케이터가 실제와 다르게 표시 권장: `period`, `sortMode`, `investedOnly` 변경 시 `prevRankMap.current = new Map()` 호출 심각도: Low

E. DB 쿼리/서버 성능 — 2건

[DB-M-01] 관리자 체결 감사 — 사용자명 enrichment N+1 패턴 (Medium)

현상: `order-proxy.controller.ts:162-179` 에서 체결 내역 조회 후 사용자명을 enrichment하기 위해 `POST /users/by-ids` 호출. 현재는 한번의 배치 호출이지만, 체결 내역의 `userId`가 다수(100+)일 경우 대량 ID 배열 전송 위치: `backend/services/api-gateway/src/proxy/order-proxy.controller.ts:162-179` 영향: 관리자 페이지 체결 내역 조회 시 추가 네트워크 홉, `userId` 100개+ 시 요청 크기 증가 권장: `order-engine` DB에 사용자명 비정규화 저장 또는 Redis 캐싱으로 enrichment 최적화 심각도: Medium

[DB-L-01] 거래 내역(transactions) 조회 — limit=200 하드코딩 (Low)

현상: `useTransactions(limit = 200)` 훅에서 기본 200건을 가져옴. `PortfolioHistoryChart` 에서 사용하지만, 기간 필터('1W' 등)가 적용되어도 항상 200건을 전부 가져온 후 클라이언트에서 기간 필터링 수행 위치: `frontend/src/hooks/usePortfolio.ts:240` 영향: 1주 조회 시에도 불필요한 90일+ 거래 데이터 전송 권장: 기간에 따라 `limit` 또는 `since` 파라미터를 동적으로 조정 심각도: Low

F. WebSocket/실시간 처리 — 2건

[WS-M-01] useWebSocket — symbols 배열 변경 시 전체 목록 재전송 (Medium)

현상: `useWebSocket.ts:196-198` 에서 `symbols` 변경 시 `socket.emit('subscribe', { symbols })` 로 전체 심볼 목록을 재전송. diff 기반으로 `removed` 심볼만 `unsubscribe`하지만, `subscribe`는 항상 전체 목록을 전송하여 서버에서 중복 처리가 필요 위치: `frontend/src/hooks/useWebSocket.ts:194-198` 영향: 심볼 100개+ 시 불필요한 전체 목록 재전송 권장: `subscribe` 이벤트도 diff 기반으로 신규 추가된 심볼만 전송하거나, 서버가 이미 구독 중인 심볼을 중복 무시하는지 확인 심각도: Medium

[WS-L-01] 종목 상세 — 단일 심볼에 배열 래핑 오버헤드 (Low)

현상: `asset/[symbol]/page.tsx:146` 에서 `useWebSocket([symbol], handlePriceUpdate)` 로 단일 심볼을 배열로 래핑하여 전달. 내부적으로 `prevSymbolsRef` 비교, `symbols.includes()` 호출 등이 배열 기반으로 동작하여 단일 심볼에 불필요한 배열 연산 발생 위치: `frontend/src/app/(main)/asset/[symbol]/page.tsx:146` 영향: 미미하지만 `useSingleSymbolWebSocket` 같은 경량 전용 훅이 있으면 불필요한 배열 연산 제거 가능 권장: 단일 심볼 전용 훅 제공 또는 현재 구조 유지 (실질적 영향 미미) 심각도: Low

종합 평가

8차 성능 감사에서 총 16건의 이슈를 발견하였습니다. 7차 대비 커뮤니티 페이지 코드 스플리팅(PF-M-01), TipTap 서버 업로드 전환(IMG-M-01), 주문/체결 서버 사이드 필터(API-M-01/02) 등 4건이 수정되었고, AssetList 심볼 비교 최적화(PF-M-04)가

부분 수정되었습니다.

가장 시급한 개선 항목은:

1. **AI 분석 뉴스 200건 과잉 조회** (PF-H-01) — 서버 측 날짜 필터 적용으로 네트워크/서버 부하 제거
2. **뉴스 필터 200건 일괄 조회** (API-H-01) — 서버 측 검색/날짜 필터 파라미터 추가
3. **대시보드 단일 컴포넌트** (BD-M-01) — AI 모달 코드 스플리팅으로 초기 번들 축소

전체적으로 프론트엔드의 "전체 조회 + 클라이언트 필터" 패턴이 뉴스 영역에 집중적으로 남아있어, 이 부분의 서버 측 필터링 전환이 가장 효과적인 성능 개선이 될 것입니다. 주문/체결 영역은 서버 필터 지원이 추가되어 점진적으로 개선되고 있습니다.